

P2P File Transfer System

Comprehensive Technical Report

Table of Contents

1. Project Overview
 2. System Architecture
 3. Implementation Details
 4. Functional Components
 5. Non-Functional Components
 6. Technology Stack
 7. Achievements & Challenges
 8. Testing & Validation
 9. Conclusion
-

1. Project Overview

1.1 Objective

Develop a decentralized peer-to-peer file transfer system that enables efficient file sharing across local area networks without relying on centralized cloud services.

1.2 Scope

- File chunking and distribution

- Peer-to-peer chunk transfer
- Metadata management
- Multiple transfer protocols (HTTP/TCP)
- Web-based user interfaces
- Token-based authentication

1.3 Target Use Cases

- Educational institutions for assignment distribution
 - Office environments for document sharing
 - Collaborative teams for large file exchange
 - Local network file distribution
-

2. System Architecture

2.1 Architecture Pattern

Hybrid P2P Architecture

- Combines centralized tracker (for coordination) with decentralized storage (P2P)
- Tracker maintains registry but does not store all files
- Peers contribute storage and bandwidth

2.2 Core Components

Privileged Peer (Tracker)

- **Role:** Central coordinator and initial file source

- **Responsibilities:**
 - Peer registration and authentication
 - File metadata registry
 - Chunk location tracking
 - Initial file hosting
- **Port:** 8000 (HTTP)

Peer Nodes

- **Role:** Distributed clients for file sharing
- **Responsibilities:**
 - Download files from network
 - Serve chunks to other peers
 - Announce chunk availability
 - Local storage management
- **Ports:** 9000+ (HTTP), 9001+ (TCP)

2.3 Data Flow Architecture

Upload Flow:

Admin → Chunker → Storage → Tracker Registry

Download Flow:

Peer → Tracker (metadata) → Tracker/Peers (chunks) → Local Storage → Reassembly

Direct Transfer Flow:

Peer A → TCP Connection → Peer B (metadata + chunks)

3. Implementation Details

3.1 File Chunking Implementation

Location: `privileged_peer/chunker.py`, `shared/chunker.py`

Process:

1. **File Reading:** Binary read of input file
2. **Chunking:** Split into 512KB (524,288 bytes) segments
3. **Hashing:** SHA256 hash calculation for each chunk
4. **Storage:** Save chunks with naming pattern `{filename}_chunk_{index}`
5. **Metadata Generation:** Create JSON with chunk details

Code Implementation:

```
python

def chunk_file(file_path: str, out_dir: str = None):
    chunks = []
    with open(file_path, "rb") as f:
        index = 0
        while True:
            data = f.read(CHUNK_SIZE) # 512KB
            if not data:
                break
            chunk_hash = sha256(data)
            chunk_name = f"{file_path.stem}_chunk_{index}"
            # Save chunk and record metadata
```

Technologies Used:

- Python `pathlib` for file operations
- `hashlib.sha256()` for hash generation
- `mimetypes` module for file type detection

Achievements:  Successfully chunks files of any size  Preserves file integrity through hashing 
Automatic MIME type detection  Handles various file formats (PDF, images, documents, videos)

Limitations:  No compression before chunking  Fixed chunk size (not adaptive)

3.2 Metadata Management

Location: `privileged_peer/metadata.py`, `peer_node/metadata.py`, `shared/metadata.py`

Metadata Structure:

json

```
{
  "original_name": "document.pdf",
  "original_extension": ".pdf",
  "file_stem": "document",
  "mime_type": "application/pdf",
  "total_chunks": 10,
  "chunks": [
    {
      "index": 0,
      "hash": "abc123...",
      "filename": "document_chunk_0",
      "size": 524288
    }
  ]
}
```

Functions Implemented:

- `save_metadata()` - Stores chunk information
- `load_metadata()` - Retrieves file metadata
- `list_available_files()` - Lists all registered files
- `get_file_metadata_by_stem()` - Searches by filename

Technologies Used:

- Python `json` module for serialization
- UTF-8 encoding for Unicode support
- Path-safe naming conventions

Achievements:  Complete metadata preservation  Extension and filename recovery  Flexible metadata retrieval methods  Safe filename handling (removes problematic characters)

3.3 Tracker Server Implementation

Location: `privileged_peer/server.py`

Framework: FastAPI

Endpoints Implemented:

Endpoint	Method	Purpose	Authentication
<code>/join</code>	POST	Peer registration	None (generates token)
<code>/announce_chunk</code>	POST	Chunk availability announcement	Token required
<code>/metadata/{file_stem}</code>	GET	Retrieve file metadata	Token required
<code>/chunk/{file_stem}/{index}</code>	GET	Download specific chunk	Token required
<code>/peers/{file_stem}/{index}</code>	GET	Get chunk owner list	Token required
<code>/files</code>	GET	List all available files	Public
<code>/register_file</code>	POST	Register new file	Public (admin)
<code>/admin/peers</code>	GET	List connected peers	Public (admin)

Data Structures:

```
python
```

```
approved_peers: Dict[str, PeerInfo]    #peer_id -> PeerInfo
peer_tokens: Dict[str, str]            #peer_id -> token
chunk_locations: Dict[str, Dict[int, Set[str]]] #file -> chunk -> peers
file_registry: Dict[str, FileMetadata] #file_stem -> metadata
```

Technologies Used:

- FastAPI for REST API
- Pydantic for data validation
- `secrets` module for token generation
- Uvicorn ASGI server

Achievements: ✓ RESTful API design ✓ Token-based authentication ✓ Dynamic chunk location tracking ✓ Metadata caching on startup ✓ URL encoding handling for special characters

Functional Aspects:

- Peer registration with unique IDs
- Secure token generation (32-byte URL-safe)
- Chunk location updates in real-time
- Supports multiple peers serving same chunk
- Automatic tracker self-registration as chunk source

3.4 Peer Client Implementation

Location: `peer_node/peer_client.py`

Class: `PeerClient`

Key Methods:

Method	Purpose	Returns
<code>join_network()</code>	Register with tracker	Boolean (success)
<code>get_metadata(file_stem)</code>	Fetch file metadata	Dict or None
<code>list_files()</code>	Get available files	List of file info
<code>find_chunk_owners(file_stem, index)</code>	Locate chunk sources	List of peer info
<code>download_file(file_stem)</code>	Download complete file	Status string
<code>announce_chunk(file_stem, index)</code>	Tell tracker about chunk	None
<code>reassemble(file_stem, metadata, chunks)</code>	Merge chunks into file	Boolean
<code>push_file_tcp(ip, port, file_stem)</code>	Direct TCP transfer	Status string

Download Logic:

1. Request metadata from tracker
2. For each chunk:
 - Check if already exists locally (skip if valid)
 - Query tracker for chunk owners
 - Try downloading from tracker first, then peers
 - Verify SHA256 hash
 - Save to local storage
 - Announce ownership to tracker
3. Reassemble all chunks

4. Save complete file to downloads folder

Technologies Used:

- `requests` library for HTTP operations
- `threading` for background server
- `hashlib` for verification
- `pathlib` for file operations

Achievements:  Automatic local chunk detection (avoids re-download)  Hash verification with retry logic  Multi-source download capability  Announces chunks as downloaded (becomes a source)  Proper error handling and fallbacks

Challenges Addressed:

- URL encoding for filenames with spaces
- Path resolution across different execution contexts
- Token management across requests
- Concurrent server and client operations

3.5 Peer Server Implementation

Location: `peer_node/peer_server.py`

Framework: FastAPI

Purpose: HTTP server for serving chunks to other peers

Endpoints:

Endpoint	Method	Purpose
<code>/chunk/{file_stem}/{chunk_index}</code>	GET	Serve chunk file
<code>/</code>	GET	Health check

Chunk Serving Logic:

1. Receive chunk request
2. Check `storage/received_chunks/` first
3. Fallback to `storage/chunks/` if not found
4. Return chunk file or 404

Technologies Used:

- FastAPI for server framework
- FileResponse for efficient file serving
- Uvicorn for ASGI serving

Achievements: ☒ Lightweight HTTP server ☒ Automatic chunk location detection ☒ Fast file serving with FileResponse ☒ Background execution (daemon thread)

3.6 TCP Direct Transfer Implementation

Location: `peer_node/tcp_handler.py`, `privileged_peer/tcp_handler.py`

Protocol Design:

Packet Structure:

Header Length (4B)	JSON Header (N)	Raw Data (M)	
Network Byte Order	UTF-8 Encoded	Binary	

Header Format:

```
json
{
  "packet_type": "metadata" | "chunk",
  "file_stem": "filename",
  "chunk_index": 0 // Only for chunks
}
```

Class: `TCPServer`

Key Methods:

- `start()` - Bind to port and start listening
- `_accept_clients()` - Accept incoming connections
- `_handle_client(conn)` - Process incoming packets
- `recv_exact(conn, n)` - Reliable byte reading

Function: `send_tcp_packet(ip, port, header, file_path)`

Process Flow:

Sender:

1. Create JSON header with packet type and metadata

2. Calculate header length
3. Send 4-byte length prefix (big-endian)
4. Send JSON header
5. Stream file data in 4KB chunks
6. Close connection

Receiver:

1. Read 4-byte header length
2. Read exact header JSON
3. Parse packet type
4. Stream data to appropriate storage location
 - Metadata → `storage/metadata/`
 - Chunks → `storage/received_chunks/`
5. Close connection

Technologies Used:

- Python `socket` module for TCP
- `struct.pack/unpack` for binary encoding
- `json` for header serialization
- `threading` for concurrent client handling

Achievements:  Custom binary protocol implementation  Reliable packet transmission 

Supports metadata and chunk transfers  Multi-threaded server for concurrent transfers  Automatic port allocation

Advantages:

- No HTTP overhead
 - Direct peer-to-peer (no tracker involvement)
 - Suitable for bulk transfers
 - Lower latency for local network
-

3.7 User Interface - Admin Dashboard

Location: `privileged_peer/dashboard.py`

Framework: Streamlit

Pages:

Page 1: Upload Files

- File uploader widget (supports PDF, DOCX, TXT, ZIP, JPG, PNG, MP4)
- Automatic chunking on upload
- Metadata display with chunk details
- File registration with tracker
- Active file registry viewer

Page 2: Peer Management

- Real-time connected peer list
- Peer connection details (ID, host, port)
- Direct TCP push functionality
 - Target IP/port input
 - File selection dropdown

- Progress bar for multi-chunk transfer
- Success/failure notifications

Technologies Used:

- Streamlit for web UI
- `requests` for API calls
- `pathlib` for file handling
- Custom TCP handler integration

Achievements:  Intuitive file upload interface  Real-time network monitoring  Direct peer-to-peer transfer controls  Visual feedback for all operations  Auto-fixing tracker URL (adds protocol and port)

UI Features:

- Expandable sections for organized layout
 - Progress bars for transfer tracking
 - Toast notifications for quick feedback
 - Color-coded status indicators
 - Responsive columns for data display
-

3.8 User Interface - Peer Dashboard

Location: `peer_node/dashboard.py`

Framework: Streamlit

Sections:

Connection Management

- Tracker URL configuration
- Auto-connect on startup
- Manual reconnect option
- Connection status display (token, IP, ports)

Network Files Browser

- Lists all available files from tracker
- Shows file details (name, chunks, MIME type)
- One-click download buttons
- Progress status with success animations

Direct TCP Transfer

- Target peer IP/port input
- File selection from available files
- Push entire file via TCP
- Transfer status feedback

Received Files Manager

- Scans for pushed metadata
- Shows chunk completion status (X/Y chunks ready)
- "Finalize & Save" button for complete files
- Automatic reassembly to downloads folder

Chunk Inventory

- Raw chunk viewer
- Total chunk count display

Technologies Used:

- Streamlit session state for client persistence
- Real-time UI updates with `st.rerun()`
- Custom peer client integration
- JSON file parsing for metadata

Achievements:  Auto-initialization of peer client  Background HTTP and TCP servers  Real-time file browsing  Pending file management (TCP received)  Progress tracking with visual feedback  Balloons animation on success

User Experience Features:

- Auto-connect on startup
 - Smart URL fixing (adds http:// and port)
 - Container borders for visual separation
 - Status badges (success/warning/info)
 - Expandable sections for advanced features
-

4. Functional Components

4.1 Fully Functional Features

File Chunking System

- **Status:**  Fully Operational
- **Implementation:** Complete in `chunker.py`
- **Capabilities:**
 - Any file size supported
 - 512KB chunk size (configurable constant)
 - SHA256 hash generation
 - MIME type auto-detection
 - Extension preservation
- **Testing:** Validated with PDF, images, documents, archives

Metadata Management

- **Status:**  Fully Operational
- **Implementation:** Complete in `metadata.py` (all modules)
- **Capabilities:**
 - JSON serialization
 - UTF-8 encoding support
 - Multiple retrieval methods (by stem, by name)
 - Safe filename handling
- **Testing:** Validated with various file types and special characters

Tracker Server

- **Status:**  Fully Operational
- **Implementation:** Complete in `server.py`
- **Capabilities:**

- Peer registration with token auth
- Chunk location tracking
- Metadata serving
- Chunk serving (as primary source)
- File registry management
- Admin endpoints
- **Testing:** Validated with multiple concurrent peers

Peer Client

- **Status:**  Fully Operational
- **Implementation:** Complete in `peer_client.py`
- **Capabilities:**
 - Network joining with authentication
 - File downloading with verification
 - Multi-source chunk retrieval
 - Automatic chunk announcement
 - Local chunk detection (avoids re-download)
 - File reassembly
- **Testing:** Validated with complete download workflows

Peer Server

- **Status:**  Fully Operational
- **Implementation:** Complete in `peer_server.py`
- **Capabilities:**
 - HTTP chunk serving

- Health check endpoint
- Background execution
- **Testing:** Validated with peer-to-peer transfers

TCP Direct Transfer

- **Status:**  Fully Operational
- **Implementation:** Complete in `tcp_handler.py`
- **Capabilities:**
 - Custom binary protocol
 - Metadata transfer
 - Chunk transfer
 - Multi-threaded receiver
 - Automatic port allocation
- **Testing:** Validated with large file transfers

Admin Dashboard

- **Status:**  Fully Operational
- **Implementation:** Complete in `privileged_peer/dashboard.py`
- **Capabilities:**
 - File upload and chunking
 - Network monitoring
 - TCP push to peers
- **Testing:** Validated end-to-end workflow

Peer Dashboard

- **Status:**  Fully Operational
- **Implementation:** Complete in `peer_node/dashboard.py`
- **Capabilities:**
 - File browsing and download
 - TCP transfer management
 - Received file handling
 - Connection management
- **Testing:** Validated user workflows

Authentication System

- **Status:**  Fully Operational
- **Implementation:** Token-based in `server.py`
- **Capabilities:**
 - Secure token generation
 - Token validation on all protected endpoints
 - Per-peer token management
- **Testing:** Validated unauthorized access prevention

Hash Verification

- **Status:**  Fully Operational
- **Implementation:** Throughout download logic
- **Capabilities:**
 - SHA256 integrity checking

- Automatic retry on mismatch
 - Pre-download verification for local chunks
 - **Testing:** Validated with corrupted chunk scenarios
-

4.2 ⚠️ Partially Functional Features

Port Management

- **Status:** ⚠️ Mostly Functional
- **Issues:**
 - Race conditions possible during port allocation
 - Inconsistent binding host parameter ("" vs specific IP)
- **Workaround:** Try-except with retry logic
- **Location:** `config.py` - `find_available_port()`

URL Encoding Handling

- **Status:** ⚠️ Functional with Edge Cases
- **Issues:**
 - Filenames with spaces require URL encoding
 - Multiple encoding attempts in different parts
- **Solution:** Using `urllib.parse.quote()` and fallback checks
- **Location:** `server.py`, `peer_client.py`

Error Messages

- **Status:** ⚠️ Basic Implementation

- **Issues:**
 - Generic error messages in some cases
 - Limited user guidance for failures
 - **Current State:** Console logging functional, UI feedback basic
-

4.3 ✗ Non-Functional / Missing Features

Resume Capability

- **Status:** ✗ Not Implemented
- **Description:** Cannot resume interrupted downloads
- **Impact:** Must restart download from beginning
- **Future Work:** Would require checkpoint metadata

Bandwidth Throttling

- **Status:** ✗ Not Implemented
- **Description:** No control over transfer speeds
- **Impact:** May saturate network
- **Future Work:** Rate limiting in chunk transfers

File Encryption

- **Status:** ✗ Not Implemented
- **Description:** Files transferred in plaintext
- **Impact:** Security concern for sensitive files
- **Future Work:** AES encryption layer

Compression

- **Status:** ✗ Not Implemented
- **Description:** No compression before chunking
- **Impact:** Larger transfer sizes
- **Future Work:** GZIP/LZMA before chunking

Parallel Downloads

- **Status:** ✗ Not Implemented
- **Description:** Sequential chunk download only
- **Impact:** Slower than possible
- **Future Work:** Concurrent chunk downloads

WAN Support

- **Status:** ✗ Not Implemented
- **Description:** LAN-only operation
- **Impact:** Cannot work over Internet
- **Future Work:** NAT traversal, port forwarding

DHT (Distributed Hash Table)

- **Status:** ✗ Not Implemented
- **Description:** Requires central tracker
- **Impact:** Single point of failure for coordination
- **Future Work:** Tracker-less operation

File Search

- **Status:** ✗ Not Implemented
- **Description:** Must browse full file list
- **Impact:** Difficult with many files
- **Future Work:** Search by filename, type, size

User Access Control

- **Status:** ✗ Not Implemented
- **Description:** All peers see all files
- **Impact:** No privacy controls
- **Future Work:** Permission-based file sharing

Transfer Statistics

- **Status:** ✗ Not Implemented
 - **Description:** No bandwidth/speed metrics
 - **Impact:** Limited visibility into performance
 - **Future Work:** Real-time speed display, transfer history
-

5. Technology Stack

5.1 Programming Language

- **Python 3.8+**
 - Used throughout entire project

- Chosen for: Rich library ecosystem, rapid development, cross-platform

5.2 Web Frameworks

FastAPI

- **Version:** Latest (from requirements)
- **Used In:** `server.py`, `peer_server.py`
- **Purpose:** REST API implementation
- **Why Chosen:**
 - High performance (ASGI)
 - Automatic API documentation (Swagger/OpenAPI)
 - Built-in validation with Pydantic
 - Easy async support (though not used here)

Streamlit

- **Version:** Latest (from requirements)
- **Used In:** `dashboard.py` (both peer and admin)
- **Purpose:** Web-based user interfaces
- **Why Chosen:**
 - Rapid prototyping
 - Python-native (no HTML/CSS/JS needed)
 - Built-in widgets for file upload, forms
 - Session state management
 - Real-time updates

5.3 HTTP Client

- **Requests Library**
 - **Used In:** `peer_client.py`, `dashboard.py`
 - **Purpose:** HTTP API calls to tracker and peers
 - **Features Used:**
 - GET/POST requests
 - JSON payload handling
 - Query parameters
 - Timeout configuration
 - Binary content handling

5.4 ASGI Server

- **Uvicorn**
 - **Used In:** Server execution
 - **Purpose:** Run FastAPI applications
 - **Features Used:**
 - Host/port binding
 - Log level configuration
 - Programmatic server start

5.5 Data Validation

- **Pydantic**
 - **Used In:** `config.py` models
 - **Purpose:** Data structure validation

- **Models Defined:**
 - PeerInfo
 - ChunkLocation
 - FileMetadata
 - ChunkData

5.6 Standard Library Modules

Module	Used In	Purpose
<code>hashlib</code>	<code>chunker.py</code>	SHA256 hash generation
<code>json</code>	Multiple	JSON serialization/deserialization
<code>pathlib</code>	Multiple	Path operations (cross-platform)
<code>socket</code>	<code>config.py</code> , <code>tcp_handler.py</code>	Network operations, IP detection, TCP
<code>threading</code>	<code>peer_client.py</code> , <code>tcp_handler.py</code>	Background servers, concurrent handling
<code>struct</code>	<code>tcp_handler.py</code>	Binary data packing (big-endian)
<code>secrets</code>	<code>server.py</code>	Cryptographically secure tokens
<code>mimetypes</code>	<code>chunker.py</code>	File type detection
<code>logging</code>	Multiple	Logging framework
<code>time</code>	Various	Timestamps, delays
<code>subprocess</code>	<code>launcher.py</code> , <code>test_network.py</code>	Process management

5.7 Development Tools

Launcher System

- **File:** `launcher.py`
- **Purpose:** Unified entry point for components
- **Implementation:** Menu-driven subprocess execution

Diagnostic Tools

- **File:** `diagnose_server.py`, `debug_space.py`
 - **Purpose:** Server endpoint testing, file existence debugging
-

6. Achievements & Challenges

6.1 Major Achievements

✓ Complete P2P Implementation

- Successfully implemented all core P2P functionalities
- Decentralized storage with centralized coordination
- Functional file transfer with integrity guarantees

✓ Dual Protocol Support

- HTTP for standard requests (RESTful)
- TCP for direct transfers (custom protocol)
- Appropriate use of each protocol

✓ **User-Friendly Interfaces**

- No command-line expertise required
- Visual feedback for all operations
- Intuitive workflow design

✓ **Robust Error Handling**

- Hash verification with retry
- Graceful degradation on peer unavailability
- Connection error recovery

✓ **Flexible Architecture**

- Modular code organization
- Reusable components across modules
- Easy to extend

6.2 Technical Challenges Overcome

Challenge 1: Path Resolution

Problem: Different execution contexts (dashboard vs CLI) caused path errors

Solution:

```
python

BASE_DIR = Path(__file__).resolve().parent.parent
STORAGE_PATH = BASE_DIR / "storage"
```

- Absolute path resolution from script location
- Consistent across all execution methods

Challenge 2: URL Encoding for Filenames

Problem: Filenames with spaces broke URL paths

Solution:

```
python

from urllib.parse import quote, unquote
safe_stem = quote(file_stem)
# In server: try both encoded and decoded versions
```

- URL encoding in client requests
- Fallback checks in server for both versions

Challenge 3: Port Conflicts

Problem: Multiple peers on same machine needed unique ports

Solution:

```
python

def find_available_port(start_port, host):
    for port in range(start_port, max_port):
        try:
            s.bind((host, port))
            return port
        except OSError:
            continue
```

- Dynamic port allocation
- Range scanning starting from base port

Challenge 4: Concurrent Operations

Problem: Client needs to download while serving chunks

Solution:

- Background threading for servers
- Daemon threads that don't block main program
- Separate HTTP and TCP servers

Challenge 5: Metadata File Detection

Problem: Metadata lookup failed with special characters

Solution:

```
python

# Try multiple lookup strategies:
# 1. Exact match
# 2. URL decoded match
# 3. Glob search through directory
# 4. Stem matching inside JSON files
```

6.3 Lessons Learned

Architecture

- Hybrid P2P (tracker + distributed) is practical for LAN

- Centralized coordination simplifies peer discovery
- Storage redundancy increases availability

Protocol Design

- Custom TCP protocol needs careful specification
- Header-first design enables flexible payloads
- Network byte order crucial for cross-platform

User Interface

- Streamlit excellent for rapid prototyping
- Session state critical for stateful apps
- Progress feedback significantly improves UX

Error Handling

- Hash verification essential for integrity
 - Retry logic should be automatic
 - User shouldn't see technical errors
-

7. Testing & Validation

7.1 Test Scenarios Executed

Functional Testing

Test Case	Description	Result
File Upload	Upload various file types	✓ Pass
Chunking	Files split correctly into 512KB chunks	✓ Pass
Hash Generation	Unique hashes for each chunk	✓ Pass
Metadata Creation	Complete metadata with all fields	✓ Pass
Tracker Registration	Peer successfully joins network	✓ Pass
Token Authentication	Unauthorized requests rejected	✓ Pass
File Listing	All registered files displayed	✓ Pass
Chunk Download	Download from tracker	✓ Pass
Peer Download	Download from another peer	✓ Pass
Hash Verification	Corrupted chunks rejected	✓ Pass
Reassembly	Chunks merged correctly	✓ Pass
Extension Preservation	Original filename restored	✓ Pass
TCP Transfer	Direct peer-to-peer push	✓ Pass
Multiple Peers	3+ peers simultaneously	✓ Pass
Chunk Announcement	Peers become sources after download	✓ Pass

File Type Testing

File Type	Extension	Size	Result
PDF Document	.pdf	2.3 MB	✓ Pass
Image	.jpg	1.5 MB	✓ Pass
Image	.png	800 KB	✓ Pass
Text	.txt	50 KB	✓ Pass
Archive	.zip	5 MB	✓ Pass
Document	.docx	1.2 MB	✓ Pass

Edge Case Testing

Test Case	Description	Result
Filename with Spaces	"my document.pdf"	✓ Pass (URL encoding)
Large File	50+ MB file	✓ Pass (100+ chunks)
Small File	< 512KB (single chunk)	✓ Pass
Special Characters	File with &, #, %	⚠ Partial (some chars fail)
Simultaneous Downloads	2 peers downloading same file	✓ Pass
Peer Disconnect	Peer goes offline during download	✓ Pass (fallback works)
Reconnection	Peer rejoins network	✓ Pass (token reuse)

7.2 Performance Observations

Chunking Speed

- 10 MB file: ~1-2 seconds
- 50 MB file: ~5-8 seconds
- Bottleneck: Disk I/O

Download Speed

- Local network: 10-50 MB/s per peer
- Limited by: Network bandwidth, single-threaded download
- Improvement potential: Parallel chunk downloads

Reassembly Speed

- Negligible overhead (< 1 second for most files)
- Sequential I/O performs well

7.3 Known Issues

Issue 1: Port Allocation Race Condition

- **Severity:** Low
- **Frequency:** Rare (multiple simultaneous starts)
- **Workaround:** Retry logic

Issue 2: Special Characters in Filenames

- **Severity:** Medium
- **Affected:** Files with /, , quotes

- **Workaround:** Filename sanitization

Issue 3: No Progress Bar During Download

- **Severity:** Low (UX issue)
 - **Impact:** User uncertainty during long downloads
 - **Status:** Basic status messages only
-

8. Code Quality & Organization

8.1 Project Structure

```
Network/
├── privileged_peer/    # Tracker/Admin component
│   ├── server.py      # FastAPI tracker server
│   ├── dashboard.py   # Admin Streamlit UI
│   ├── chunker.py     # File chunking logic
│   ├── metadata.py    # Metadata management
│   ├── tcp_handler.py # TCP server/client
│   └── config.py      # Configuration & models
│
├── peer_node/         # Peer component
│   ├── peer_client.py # Main peer client class
│   ├── peer_server.py # HTTP chunk server
│   ├── dashboard.py   # Peer Streamlit UI
│   ├── tcp_handler.py # TCP server/client
│   ├── metadata.py    # Metadata management
│   └── config.py      # Configuration & models
│
└── shared/            # Shared utilities (legacy)
```

| | — chunker.py
| | — metadata.py
|